

AI-Powered Incident Response Platform

Architecture, Design Decisions & Technical Reference

SDE-2 Portfolio Project • Multi-Agent • Kafka • MCP • pgvector

Executive Summary

What this platform does and why it matters

Mission: Reduce production incident MTTR (Mean Time To Resolve) from ~45 minutes to ~8 minutes by autonomously triaging, diagnosing, and remediating incidents using three specialized AI agents — without human intervention for low and medium risk fixes.

The Problem

Production incidents cost organizations an average of \$5,600 per minute in lost revenue, engineer time, and customer trust. The traditional workflow is manual: an alert fires, PagerDuty pages an on-call engineer, the engineer spends 20 minutes understanding what is broken, another 15 minutes diagnosing the root cause, and finally executes a fix. The total time from alert to resolution averages 45 minutes — most of which is cognitive work that AI can do in seconds.

The Solution

An event-driven, multi-agent system where Kafka ingests alerts from Prometheus, Grafana, and PagerDuty, and three AI agents handle the full incident lifecycle:

- **Triage Agent** — classifies severity (P1–P4), retrieves the most relevant runbook using pgvector semantic search over past incidents and runbooks.
- **Diagnosis Agent** — calls MCP tool servers (Kubernetes, database, logs, metrics) to gather diagnostic evidence and identify the root cause.
- **Remediation Agent** — creates a remediation plan, routes each action through a risk classifier, requests human approval where needed, and executes the fix.

Primary KPI	MTTR — target reduction from ~45 min to ~8 min
AI Model	Claude claude-opus-4-5 via Anthropic SDK
Tool Protocol	MCP (Model Context Protocol) — synchronous tool calls
Event Bus	Apache Kafka 3.5 — alert ingestion and agent handoffs only
Vector Memory	PostgreSQL + pgvector — runbook and incident embeddings
Language	Python 3.12, async throughout (FastAPI, asyncpg, redis.asyncio)

System Architecture

Components, data flows, and design decisions

End-to-End Data Flow

An alert enters the system and passes through three agents in strict sequence. Each agent enriches the central IncidentContext object and publishes it to the next Kafka topic. The agents themselves call tools synchronously via MCP — not through Kafka.

Step	Component	Input	Output	Kafka Topic
1	Alert Ingestor	Prometheus / Grafana / Webhook	AlertEvent (normalized)	alerts.raw
2	Triage Agent	AlertEvent	IncidentContext + severity + runbook	alerts.triaged
3	Diagnosis Agent	IncidentContext	Root cause + evidence list	incidents.active
4	Remediation Agent	IncidentContext + RCA	Actions executed + resolution	incidents.resolved
5	Audit Consumer	All audit events	Postgres row + Slack message	—
6	Learning Loop	Resolved incident	pgvector embedding upserted	—

Why Three Agents Instead of One

The three-agent design is driven by three engineering principles, not convenience:

- **Permission isolation:** The Triage and Diagnosis agents have zero write access to infrastructure. Only the Remediation agent can call remediation-mcp — and only after human approval for medium/high risk actions. A single agent would need write access during diagnosis, violating the principle of least privilege.
- **Failure isolation:** If the Diagnosis agent times out (Loki is down), the triage result is already persisted in Kafka. The incident can be retried at the diagnosis step without re-running triage. In a single-agent system, the entire pipeline restarts.
- **Auditability:** Each agent handoff creates a discrete Kafka event. You can audit the exact triage decision, the exact root cause the diagnosis agent reached, and the exact remediation plan — as separate records, not buried in an LLM reasoning chain.

Why Kafka is Used — and Where It Is Not

Kafka IS used for: Alert ingestion (bursty push events), agent pipeline handoffs (durable, replayable), and audit event fan-out (multiple independent consumers).

Kafka is NOT used for: Agent-to-tool communication. Agents call MCP tool servers directly via HTTP because they need synchronous results to reason. Adding Kafka to tool calls would add 20-200ms latency with zero benefit — the agent blocks waiting for the result either way.

Kafka Topics & Configuration

Topic Reference

Topic	Producer	Consumer(s)	Message Type
alerts.raw	alert-ingestor	triage-agent	AlertEvent (JSON)
alerts.triaged	triage-agent	diagnosis-agent	IncidentContext (JSON)
incidents.active	diagnosis-agent	remediation-agent	IncidentContext (JSON)
audit.events	all agents	audit-consumer (fan-out)	Audit event dict
incidents.resolved	remediation-agent	learning-loop	IncidentContext (JSON)

Key Kafka Settings and Why

- **num_partitions: 4** — four parallel consumers per topic. If you run 4 instances of the triage-agent, Kafka distributes one partition to each. Horizontal scaling with zero config changes.
- **enable.auto.commit: false** — Kafka commits offsets (marks messages as processed) manually, only after the handler completes successfully. If the agent crashes mid-analysis, the message is redelivered. No lost incidents.
- **max.poll.interval.ms: 300000** — 5-minute timeout before Kafka considers a consumer dead. LLM calls can take 30-60 seconds; without this, Kafka kicks the consumer out of the group mid-processing.
- **acks: all** — producer waits for all in-sync replicas to confirm before returning. In a multi-broker production cluster, this guarantees no message loss on broker failure.
- **log.retention.hours: 168** — messages retained for 7 days. If the diagnosis service is down for a day, it replays all missed alerts when it restarts.

MCP Tool Servers

What each server exposes and which agents can call it

Each MCP server is an independent FastAPI microservice using the fastmcp library. Claude discovers available tools automatically from these servers and decides which ones to call based on the diagnostic task at hand.

Server	Port	Agent Access	Read/Write	Tools Exposed
k8s-mcp	8001	Diagnosis only	Read only	get_pod_status, get_pod_logs, get_recent_events, get_deployment_history, describe_service, get_resource_usage
db-mcp	8002	Diagnosis only	Read only	get_connection_count, get_slow_queries, get_replication_lag, check_table_bloat, get_recent_errors, get_lock_waits
logs-mcp	8003	Diagnosis only	Read only	search_logs, get_error_rate, get_stack_traces, correlate_by_trace_id, get_log_volume
metrics-mcp	8004	Diagnosis only	Read only	query_prometheus, get_service_latency, get_error_rate_history, get_saturation_metrics, get_sli_status
remediation-mcp	8005	Remediation only	Write (gated)	restart_pod (LOW), restart_deployment (MED), scale_deployment (MED), rollback_deployment (HIGH), toggle_feature_flag (MED), clear_cache (LOW), drain_node (HIGH)

Human Approval Gate

The Remediation Agent classifies every planned action by risk level before execution. The classification determines the approval workflow:

Risk Level	Examples	Approval Flow	Timeout
LOW	restart_pod, clear_cache	Auto-execute immediately, notify in Slack after	None
MEDIUM	scale_deployment, toggle_feature_flag, restart_deployment	Post Slack approval request, wait for engineer response	5 minutes then escalate
HIGH	rollback_deployment, drain_node	Post Slack + page PagerDuty, mandatory human approval	No timeout — must respond

Slack approval message format (MEDIUM/HIGH risk):

■ P1 — payment-service: connection pool exhausted
Root cause: 847 idle connections leaking from order-service
Proposed fix: restart payment-service deployment (risk: MEDIUM, downtime: <10s)
Similar past incident: Oct 14 — same fix resolved in 4 minutes
[Approve] [Reject] [Show full RCA]

pgvector & the Learning Loop

How the system gets smarter over time

Why pgvector Instead of a Dedicated Vector Database

The incidents and runbooks already live in Postgres. Using a separate vector database (Pinecone, Weaviate, Chroma) would mean two network calls — one to the vector DB for similarity search and one to Postgres for the actual incident data — then joining the results in application code. With pgvector, a single SQL query returns both the similarity score and all incident metadata. At the scale of thousands of incidents and hundreds of runbooks, Postgres + pgvector is significantly faster and operationally simpler.

How Similarity Search Works

- The triage agent receives an AlertEvent with name, service, and annotation summary.
- It generates a 1536-dimensional embedding of the combined alert text using an embedding model.
- It runs a pgvector cosine similarity query against the runbooks table and incidents table.
- The top-3 most similar runbooks and top-3 most similar past incidents are returned.
- These are injected into Claude's context as few-shot examples before the triage decision.
- The agent's first response already contains: 'We saw this in October — root cause was X, fixed in 4 minutes by restarting the app server.'

The HNSW Index

Exact nearest-neighbor search is $O(n)$ — it compares the query vector against every stored embedding. At 10,000 past incidents, this takes hundreds of milliseconds. The HNSW (Hierarchical Navigable Small World) graph index reduces this to approximately $O(\log n)$ by building a multilayer graph where each node links to its nearest neighbors. The index is created with:

```
CREATE INDEX idx_incidents_embedding ON incidents
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 128);
```

Learning Loop — Postmortem to Embedding

After every resolved incident, the learning-loop service is triggered via the incidents.resolved Kafka topic:

- **Step 1:** Engineer writes or agent auto-generates a postmortem: alert signature, root cause, what fixed it.
- **Step 2:** The learning-loop service generates an embedding of: '{alert_name} {service} {root_cause} {resolution}'.
- **Step 3:** The embedding is stored in the incidents.embedding column via pgvector.
- **Step 4:** The next similar alert finds this incident in similarity search.
- **Step 5:** The system's accuracy and speed improve with every incident resolved — no retraining required.

Data Models — Complete Reference

AlertEvent

The canonical normalized alert. Every alert source — Prometheus, Grafana, PagerDuty, Datadog, manual — is normalized into this shape at the ingestor boundary. Downstream services never deal with source-specific formats.

Field	Type	Description
alert_id	str (uuid4)	Auto-generated unique identifier
source	AlertSource enum	prometheus grafana pagerduty datadog webhook manual
status	AlertStatus enum	firing resolved silenced
severity	Severity None	None until triage agent sets it (P1/P2/P3/P4)
name	str	Alert name: 'HighErrorRate', 'PodCrashLooping'
service	str	Affected service: 'payment-service', 'order-api'
environment	str	Default 'production'
labels	dict[str, str]	{'env':'prod','team':'payments'}
annotations	AlertAnnotation	summary, description, runbook_url
raw_payload	dict	Original source JSON — preserved for debugging
fired_at	datetime	When the alert condition triggered
trace_id	str None	OpenTelemetry trace ID — added in Phase 8

Key method: `fingerprint()` returns `{'name':{service}:{environment}}` — used for Redis deduplication to prevent duplicate incidents from flapping alerts.

IncidentContext

The central object that flows through the entire pipeline. Think of it as the incident's complete memory — every agent reads it, adds its findings, and publishes the enriched version to the next Kafka topic. Fields are grouped by which agent sets them.

Field	Set By	Description
incident_id	auto	uuid4 — primary key throughout the system
status	each agent	triaging → diagnosing → remediating → resolved
alert	ingestor	The original AlertEvent — never mutated
severity	triage	P1/P2/P3/P4 classification
triage_summary	triage	Human-readable explanation of what is happening
matched_runbook_id	triage	Runbook ID from pgvector similarity search
similar_incident_ids	triage	Past incident IDs used as context
triage_confidence	triage	0.0–1.0 confidence score from Claude
root_cause	diagnosis	Root cause string from diagnosis agent

affected_services	diagnosis	List of services impacted
evidence	diagnosis	List of Evidence objects from MCP tool calls
remediation_plan	remediation	List of Action objects — planned before execution
human_approved	remediation	True after Slack approval received
approved_by	remediation	Slack user ID of approver
resolved_at	remediation	Timestamp — used to calculate MTTR
trace_id	ingestor	OpenTelemetry trace ID propagated through all stages

Infrastructure & Deployment

Docker Compose Services

Service	Image	Port(s)	Purpose
zookeeper	confluentinc/cp-zookeeper:7.5.3	2181	Kafka control plane — leader election
kafka	confluentinc/cp-kafka:7.5.3	9092 (internal) 29092 (external)	Message broker
schema-registry	confluentinc/cp-schema-registry:7.5.3	8081	Message schema validation
kafka-ui	provectuslabs/kafka-ui	8080	Visual broker management dashboard
redis	redis:7.2-alpine	6379	Dedup keys, incident state, approval tokens
postgres	pgvector/pgvector:pg16	5432	Incidents, runbooks, audit log, embeddings
prometheus	prom/prometheus:v2.48.0	9090	Metrics scraping and alert rules
grafana	grafana/grafana:10.2.0	3000	MTTR dashboards and incident views

Kafka Dual Listener Configuration

Kafka advertises its address to clients. Services inside Docker use `kafka:9092` (internal Docker network). When you run local scripts or tests from your machine, they use `localhost:29092` (mapped out of Docker). Both point to the same Kafka broker — two listener names, one broker, two audiences.

Postgres Schema

The `init.sql` file runs automatically on first container start. It creates four tables, `pgvector` HNSW indexes, and seeds three starter runbooks.

incidents

One row per incident. Contains full `IncidentContext` as JSONB, plus indexed fields for fast queries by `status/service/severity`. The embedding column is populated after resolution by the learning loop.

runbooks

Manually written or auto-generated runbooks with steps stored as JSONB. Embedding populated on creation for similarity search.

audit_events

Every agent action: `alert_received`, `runbook_found`, `pod_restarted`, etc. Referenced to `incident_id`. Used for compliance audit and incident postmortems.

approval_requests

Pending, approved, rejected, and expired approval requests. Slack message timestamp stored for live message updates when engineer approves/rejects.

Build Phases — Step-by-Step Roadmap

Phase	What Gets Built	Key Concepts Learned	Time
1	Monorepo scaffold, Docker Compose (Kafka + Postgres + Redis), shared Pydantic models	Monorepo structure, pgvector schema, Kafka config, Pydantic BaseSettings	2 days
2	Alert ingestor — FastAPI endpoints for Prometheus, Grafana, webhooks	Alert normalization, Redis fingerprint dedup, Kafka producer, manual offset commit	1 day
3	Triage Agent — Claude + pgvector runbook retrieval	Embedding generation, cosine similarity search, HNSW index, agentic calls	3 days
4	Four diagnostic MCP servers (k8s, db, logs, metrics)	fastmcp library, Kubernetes Python SDK, async HTTP with httpx	4 days
5	Diagnosis Agent — multi-turn MCP tool calling loop	Agentic loops, evidence collection, tool-use content blocks, context management	3 days
6	Remediation Agent + human approval gate via Slack	Risk classification, Slack Bolt SDK, approval token patterns in Redis	3 days
7	Audit consumer — Postgres persistence + Slack notifications	Kafka fan-out consumer groups, Slack message formatting, async SQLAlchemy	2 days
8	Learning loop — postmortem to pgvector embeddings	Embedding generation pipeline, pgvector upsert, async batch processing	2 days
9	OpenTelemetry — trace propagation through Kafka + agents	OTel SDK, context propagation, Kafka header injection, Grafana Tempo	2 days
10	Dashboard (WebSocket), CI/CD (GitHub Actions), Kubernetes manifests	FastAPI WebSocket, HPA auto-scaling, GitHub Actions pipeline, K8s secrets	3 days

Technology Stack — With Justification

Every technology was chosen to solve a specific problem. Understanding the 'why' is more important than the 'what' — interviewers ask about tradeoffs, not just tool names.

Component	Technology	Why This, Not the Alternative
AI agents	Python + anthropic SDK	Native MCP support; tool-use is first-class, not an afterthought
MCP servers	fastmcp library	10-line MCP server; Claude discovers tools automatically
Message broker	Apache Kafka 3.5	Alert bursts need buffering; durable handoffs; fan-out consumers
Vector memory	PostgreSQL + pgvector	No separate infra; join vector results with relational data in one query
Session state	Redis (async)	Sub-millisecond dedup checks; TTL-based approval tokens; pub/sub for approvals
Persistence	PostgreSQL 16	JSONB for flexible IncidentContext; pgvector for embeddings; ACID compliance
HTTP client	httpx (async)	Native async; connection pooling; cleaner API than aiohttp
Observability	OpenTelemetry + Grafana Tempo	Vendor-neutral; trace_id through Kafka headers to every service
Containers (dev)	Docker Compose	One command starts full infra; healthchecks prevent race conditions
Containers (prod)	Kubernetes + HPA	Auto-scale consumers by Kafka consumer lag metric
CI/CD	GitHub Actions	Native to GitHub; matrix builds per service; zero infra to manage
Logging	structlog (JSON)	Searchable fields; same format in every service; trace_id injection

Interview Guide

How to present this project — answers for common questions

90-Second Opening (Memorize This)

"Production incidents cost an average of \$5,600 per minute. I built a platform that reduces MTTR from 45 minutes to ~8 minutes by having three specialized AI agents autonomously handle the full incident lifecycle. The interesting engineering challenge was deciding what to automate and what to keep human-controlled. Low-risk fixes — restart a pod, clear a cache — execute automatically. High-risk actions — rollback a deployment, drain a node — always require explicit human approval via Slack, no matter how confident the agent is. The system learns from every resolved incident: embeddings of past root causes and resolutions are stored in pgvector, so the triage agent's first response to a new alert already contains relevant historical context."

Anticipated Questions and Answers

Q: Why Kafka? Couldn't you use a simple task queue like Celery?

A: Kafka gives you three things Celery can't: durable, replayable message history (so if the diagnosis service is down for an hour, no alerts are lost when it restarts); true fan-out consumers (the audit consumer, learning loop, and dashboard all independently consume the same audit.events topic with zero coupling); and built-in backpressure (if all three triage-agent instances are busy, Kafka queues the alerts rather than dropping them). The key thing I did NOT use Kafka for is agent-to-tool communication — that stays direct HTTP because the agent needs synchronous results to reason.

Q: Why three agents instead of one big agent?

A: Primarily for permission isolation. The diagnosis agent only has read access to Kubernetes, logs, and metrics. The remediation agent has write access — but only after human approval for medium and high risk actions. If I merged them into one agent, I'd need to give it write access during the diagnosis phase, which violates least privilege. The secondary benefit is failure isolation: if diagnosis times out because Loki is down, the triage result is already saved in Kafka and the incident can be retried at the diagnosis step.

Q: How does the agent avoid making things worse with autonomous remediation?

A: Three layers of safety. First, the risk classifier: every action is classified as low, medium, or high risk based on the action type and blast radius. Second, the approval gate: low-risk auto-executes, medium requires Slack approval with a 5-minute timeout, high requires explicit approval with no timeout. Third, the audit trail: every action — planned and executed — is recorded in Postgres and Kafka before execution begins, so we can always reconstruct exactly what the agent did and why.

Q: Why pgvector instead of Pinecone or a dedicated vector database?

A: The incidents and runbooks already live in Postgres. With pgvector, a single SQL query returns the similarity score AND all incident metadata in one network call. With Pinecone, I'd make a vector search call, get back IDs and scores, then make a Postgres call to fetch the actual data, then join in application code. That's two round trips, more code, and another service to operate. At the scale of thousands of incidents, pgvector with an HNSW index is fast enough — typically under 10ms for a similarity query.

Q: How does the system improve over time?

A: After every resolved incident, the learning-loop service generates an embedding of the combined alert name, root cause, and resolution summary, then stores it in the incidents table via pgvector. The next time a similar alert fires, the triage agent retrieves the top-3 most similar past incidents and injects them into Claude's context as few-shot examples. The agent then says things like 'we saw this exact pattern in October — root cause was connection pool

exhaustion, fixed by restarting the app server, took 4 minutes.' No retraining required — the memory is the database.

Quick Reference

Ports, topics, and key commands for daily development

Start the Full Stack

```
cd infra && docker compose up -d # Start all infrastructure
docker compose ps # Verify all services healthy
open http://localhost:8080 # Kafka UI – see topics and messages
open http://localhost:3000 # Grafana – MTTR dashboard
```

Send a Test Alert

```
curl -X POST http://localhost:8000/alerts/manual \
-H "Content-Type: application/json" \
-d '{"name":"HighErrorRate","service":"payment-service",'
  '"summary":"Error rate above 10% for 5 minutes"}'
```

All Service Ports

Service	Port	URL
alert-ingestor	8000	http://localhost:8000/docs
k8s-mcp	8001	http://localhost:8001/mcp
db-mcp	8002	http://localhost:8002/mcp
logs-mcp	8003	http://localhost:8003/mcp
metrics-mcp	8004	http://localhost:8004/mcp
remediation-mcp	8005	http://localhost:8005/mcp
dashboard	8006	http://localhost:8006
Kafka UI	8080	http://localhost:8080
Schema Registry	8081	http://localhost:8081
Prometheus	9090	http://localhost:9090
Grafana	3000	http://localhost:3000 (admin/admin)
Redis	6379	redis-cli -h localhost
Postgres	5432	psql -U agent_user -d incident_db -h localhost